

智能巡检车 PRO 版实验手册

雷达警卫实验

目录

1 雷达警卫	2
1.1 程序功能说明	2
1.2 程序代码参考路径	2
1.3 程序启动	2
1.3.1 启动命令	2
1.3.2 查看话题通讯节点图	3
1.4 核心源码解析	4
2 雷达跟踪实验过程	5

1

雷达警卫

1.1 程序功能说明

程序启动后, 小车会跟踪最近的一个点的目标, 当目标点横向移动的时候, 会跟目标点运动, 当目标点 靠近小车, 小于设定的距离后, 蜂鸣器会报警嘀嘀嘀响声, 直到目标点远离小车 设定的距离。

1.2 程序代码参考路径

进入 docker 容器后, 该功能源码的位置位于,

```
/root/icar_ros2_ws/icar_ws/src/icar_laser/icar_laser/las er_Warning_a1_X3.py
```

1.3 程序启动

1.3.1 启动命令

进入docker容器后, 根据实际的车型和雷达型号, 终端输入,

```
#启动小车底盘  
  
ros2 run icar_bringup Mcnamu_driver_X3  
  
#启动 A1 雷达  
  
ros2 launch slidar_ros2 slidar_launch.py
```

#启动雷达避障程序、A1 雷达

```
ros2 run icar_laser laser_Warning_a1_X3
```

1.3.2 查看话题通讯节点图

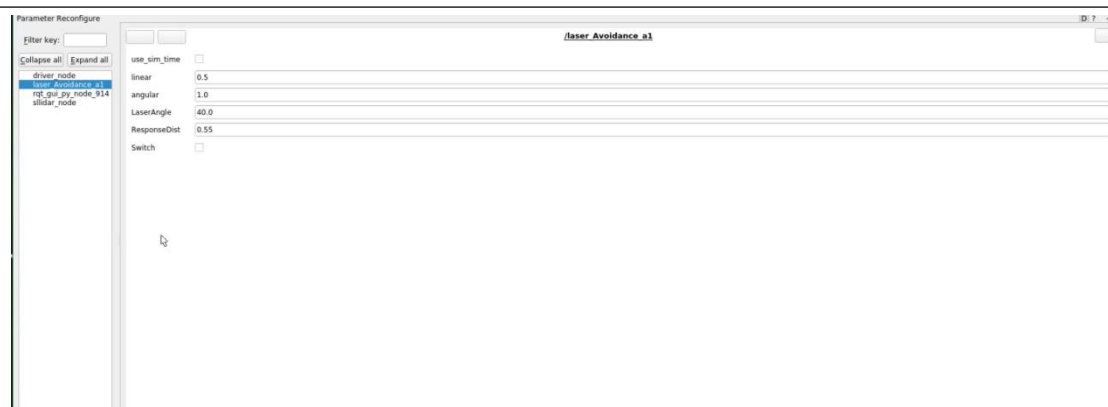
docker终端输入,

```
ros2 run rqt_graph rqt_graph
```



还可以通过动态参数调节器来设置参数的大小, 终端输入,

```
ros2 run rqt_reconfigure rqt_reconfigure
```



各参数含义如下,

参数名称	参数含义
linear	线速度
angular	角速度
LaserAngle	雷达检测角度
ResponseDist	障碍物检测距离
Switch	玩法开关

1.4 核心源码解析

以A1雷达的源码为例，主要看雷达的回调函数，这里说明了如何获取到每个角度的障碍物距离 信息，然后获取最小距离的ID，计算角速度的大小，然后最小距离与设定的距离进行比较，如果小于设 定的距离，则让蜂鸣器响，最后发布速度话题数据。

```
for i in range(len(ranges)):
    angle = (scan_data.angle_min + scan_data.angle_increment * i) * RAD2DEG
    if abs(angle) > (180 - self.LaserAngle):
        minDistList.append(ranges[i])
        minDistIDList.append(angle)
        if len(minDistList) == 0: return
    minDist = min(minDistList)
    minDistID = minDistIDList[minDistList.index(minDist)]
    if self.Joy_active or self.Switch == True:
        if self.Moving == True:
            self.pub_vel.publish(Twist())
            self.Moving = not self.Moving
            return
        self.Moving = True
    if minDist <= self.ResponseDist:
        if self.Buzzer_state == False:
            b = Bool()
            b.data = True
            self.pub_Buzzer.publish(b)
```

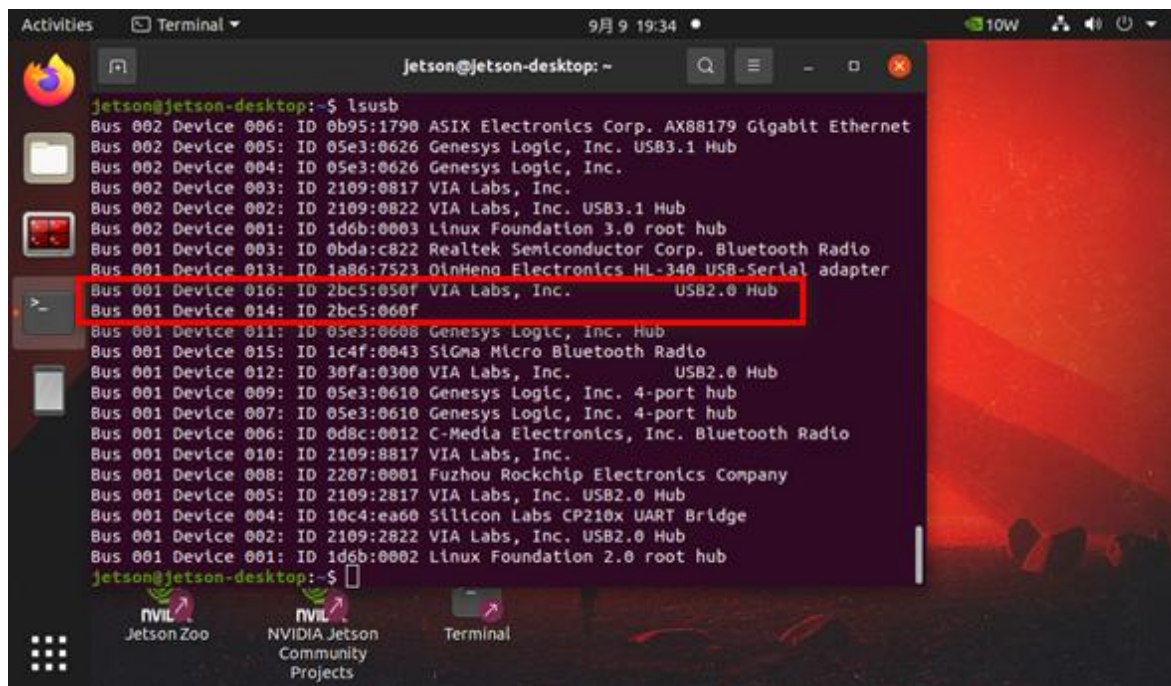
```

        self.Buzzer_state = True
    else:
        if self.Buzzer_state == True:
            self.pub_Buzzer.publish(Bool())
            self.Buzzer_state = False
    velocity = Twist()
    ang_pid_compute = self.ang_pid.pid_compute((180 - abs(minDistID)) / 36, 0)
    if minDistID > 0: velocity.angular.z = -ang_pid_compute
    else: velocity.angular.z = ang_pid_compute
    if ang_pid_compute < 0.02: velocity.angular.z = 0.0
    self.pub_vel.publish(velocity)

```

2 雷达警卫实验过程

1. 确认深度相机的编号。



观察设备编号有“2bc5”的两行，记录第四列的编号为 014、016（根据实际情况，编号有可能变化）。

异常处理：

如果只看到一行有“2bc5”，则需要重新拔插深度相机的 USB 连线，直到系统正确识别出现两行有“2bc5”。

2. 将上面的两个编号更新到~/run_docker.sh 文件中如下两行。

```

jetson@jetson-desktop: ~
#!/bin/bash
xhost +

docker run -it \
--net=host \
--env="DISPLAY" \
--env="QT_X11_NO_MITSHM=1" \
-v /tmp/.X11-unix:/tmp/.X11-unix \
-v /home/jetson/temp:/root/icar_ros2_ws/temp \
-v /home/jetson/rosboard:/root/rosboard \
-v /home/jetson/maps:/root/maps \
-v /dev/bus/usb/001/014:/dev/bus/usb/001/014 \
-v /dev/bus/usb/001/016:/dev/bus/usb/001/016 \
--device=/dev/astradept \
--device=/dev/astrauvc \
--device=/dev/video0 \
--device=/dev/myserial \
--device=/dev/rplidar \
--device=/dev/input \
-p 9090:9090 \
-p 8888:8888 \
icar/roxy:0.0 /bin/bash

"run_docker.sh" 22L, 556C
  
```

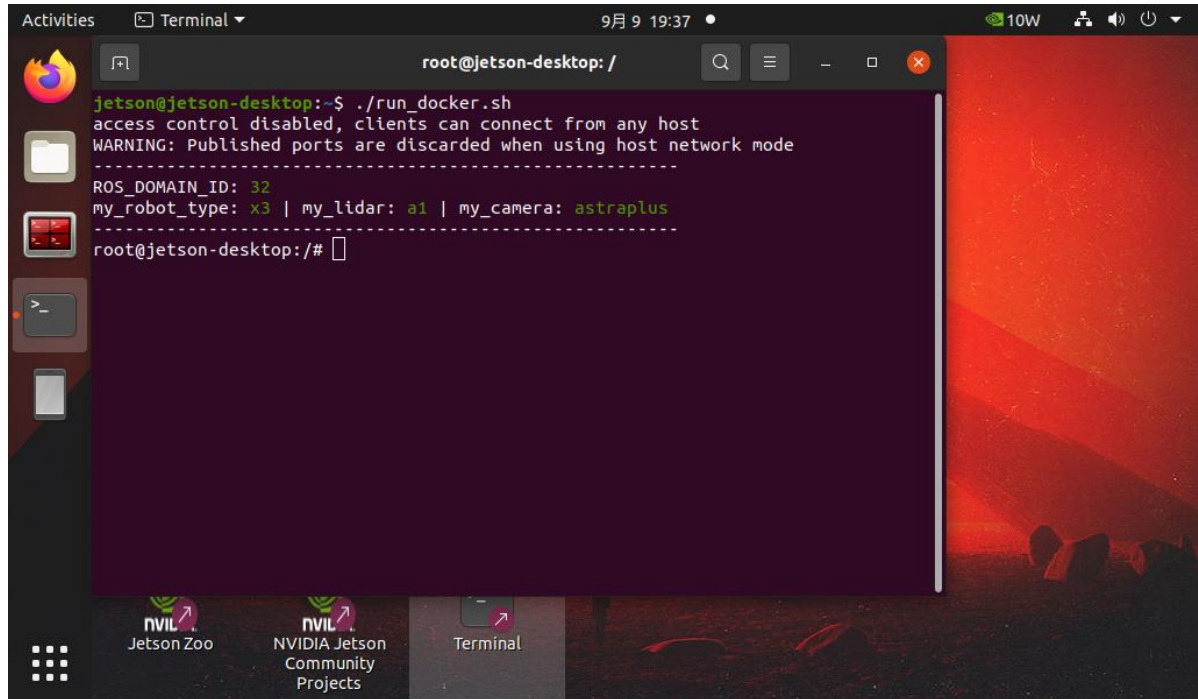
保存退出。

3. 输入指令./run_docker.sh, 启动 docker 镜像。

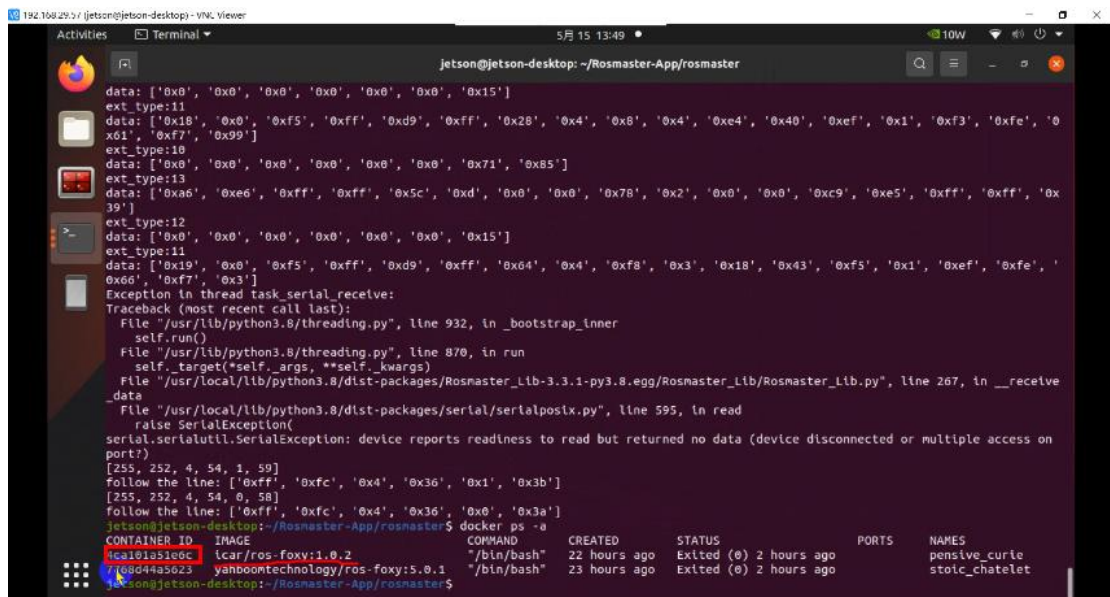
```

jetson@jetson-desktop: ~
Documents
Downloads
Maps
Music
Pictures
Public
rosboard
Rosmaster-App
Rosmaster-App-bak
run_docker.sh
software
temp
Templates
torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl
torchvision-bak
Version.txt
Videos
vision-release-0.16
vision-release-0.16.zip
yolov5-7.0
yolov5-7.0.zip
yolov5-bak
Yolov5ptFile
jetson@jetson-desktop:~$ ./run_docker.sh
  
```

如无异常, 出现如下信息, 则说明进入 docker 镜像启动成功, 并已进入 docker 环境。



4. 在新启动的 Terminal 窗口中, 使用 `docker ps -a` 指令查看 icar/ros-foxy:1.0.2 镜像容器的 ID 前缀, 比如: 这里的 4ca



5. 在 Terminal 窗口中输入 `cd` 指令进入 HOME 目录:

```

ext_type:11
data: ['0x18', '0x0', '0xf5', '0xff', '0xd9', '0xff', '0x28', '0x4', '0x8', '0x4', '0xe4', '0x40', '0xef', '0x1', '0xf3', '0xfe', '0x01', '0xf7', '0x99']
ext_type:10
data: ['0x0', '0x0', '0x0', '0x0', '0x0', '0x0', '0x71', '0x85']
ext_type:13
data: ['0xa6', '0xe6', '0xff', '0xff', '0x5c', '0xd', '0x0', '0x0', '0x78', '0x2', '0x0', '0x0', '0xc9', '0xe5', '0xff', '0xff', '0x39']
ext_type:12
data: ['0x0', '0x0', '0x0', '0x0', '0x0', '0x0', '0x15']
ext_type:11
data: ['0x19', '0x0', '0xf5', '0xff', '0xd9', '0xff', '0xd4', '0x4', '0xf8', '0x3', '0x18', '0x43', '0xf5', '0x1', '0xef', '0xfe', '0x00', '0xf7', '0x99']
Exception in thread task_serial_receive:
Traceback (most recent call last):
  File "/usr/lib/python3.8/threading.py", line 932, in _bootstrap_inner
    self.run()
  File "/usr/lib/python3.8/threading.py", line 870, in run
    self._target(*self._args, **self._kwargs)
  File "/usr/local/lib/python3.8/dist-packages/Rosmaster_Lib-3.3.1-py3.8.egg/Rosmaster_Lib/Rosmaster_Lib.py", line 267, in __receive_data
    raise SerialException(
serial.serialutil.SerialException: device reports readiness to read but returned no data (device disconnected or multiple access on port?)
[255, 252, 4, 54, 1, 59]
Follow the line: ['0xff', '0xfc', '0x4', '0x36', '0x1', '0x3b']
[255, 252, 4, 54, 0, 58]
Follow the line: ['0xff', '0xfc', '0x4', '0x36', '0x0', '0x3a']
jetson@jetson-desktop:~/Rosmaster-App/rosmaster$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
4ca101a51e6c   icar/rosmaster-foxy:1.0.2          "/bin/bash"             22 hours ago  Exited (0) 2 hours ago          pensive_curie
7768d44a5023   yahboomtechnology/ros-foxy:5.0.1   "/bin/bash"             23 hours ago  Exited (0) 2 hours ago          stoic_chatelet
jetson@jetson-desktop:~$ vi .b

```

6. 使用 vi .bashrc 编辑环境变量文件，定义 is、id、it 三条快捷指令：

```

. /usr/share/bash-completion/bash_completion
elif [ -f /etc/bash_completion ]; then
. /etc/bash_completion
fi
alias python=python3
ip=$(ip addr show wlan0 | grep -o 'inet [0-9]\+\.[0-9]\+\.[0-9]\+\.[0-9]\+' | grep -o [0-9].*)
tr [ -z $ip ]; then
ip=$(ip addr show lo | grep -o 'inet [0-9]\+\.[0-9]\+\.[0-9]\+\.[0-9]\+' | grep -o [0-9].*)
fi
echo "-----"
echo -e "MY_IP: \033[32m$ip\033[0m"
echo "-----"
export PATH=/usr/local/cuda-11.4/bin${PATH:+:${PATH}}
export LD_LIBRARY_PATH=/usr/local/cuda-11.4/lib64${LD_LIBRARY_PATH:+:${LD_LIBRARY_PATH}}
export CUDA_ROOT=/usr/local/cuda
alias ros='cd /home/jetson/Rosmaster-App/rosmaster'
alias run='/home/jetson/Rosmaster-App/rosmaster/start_app.sh'
alias start='python rosmaster_pid.py 1 50 40 2000'
alias stop='python rosmaster_pid.py 7 50 40 2000'
alias app='cd /home/jetson/Rosmaster-App/rosmaster;python3 app_sim_run.py'
alias back='if [ ! -d ~/.valid ]; then mkdir -p ~/.valid;fi;cp ~/Rosmaster-App/rosmaster/license.json ~/.valid;cp ~/Rosmaster-App/rosmaster/enc.txt ~/.valid'
alias restore='cp ~/.valid/license.json ~/Rosmaster-App/rosmaster;cp ~/.valid/enc.txt ~/Rosmaster-App/rosmaster/'
alias s='docker start 7768'
alias t='docker stop 7768'
alias d='docker exec -it 7768 /bin/bash'
alias is='docker start 4ca'
alias it='docker stop 4ca'
alias id='docker exec -it 4ca /bin/bash'
. ~/.bashrc 146L, 5048C

```

编辑完成后保存退出。

在 Terminal 窗口运行指令 source .bashrc 指令生效环境变量文件

此时，在 Terminal 窗口中可以通过 s、t、d 三个快捷指令实现对自动导航容器的启动、停止和登入操作：

快捷指令	完整指令	指令用途
is	docker start 2169	启动 ID 前缀为 2169 的 docker 容器
it	docker stop 2169	停止 ID 前缀为 2169 的 docker 容器
id	docker exec -it 2169 /bin/bash	登入 ID 前缀为 2169 的 docker 容器

7. 在 Terminal 窗口中运行快捷指令 id，进入 icar 镜像的 docker 容器：

```

root@jetson-desktop: /
[INFO] [imu_filter_madgwick_node-5]: process has finished cleanly [pid 603]
[INFO] [base_node_X3-4]: process has finished cleanly [pid 601]
[INFO] [ekf_node-0]: process has finished cleanly [pid 605]
[INFO] [joint_state_publisher-1]: process has finished cleanly [pid 595]
[ERROR] [yahboom_joy_X3-7]: process has died [pid 607, exit code -2, cmd '/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_
ctrl/lib/yahboomcar_ctrl/yahboom_joy_X3 --ros-args'].
[ERROR] [Mcnamu_driver_X3-3]: process has died [pid 599, exit code -2, cmd '/root/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomc
r_brngup/lib/yahboomcar_brngup/Mcnamu_driver_X3 --ros-args'].
[sllidar_node-8] [INFO] [1747288510.489806111] [sllidar_node]: Stop motor
[INFO] [sllidar_node-8]: process has finished cleanly [pid 615]
root@jetson-desktop: /# exit
exit
jetson@jetson-desktop: $ t
7768
jetson@jetson-desktop: $ ls
alias.txt          Pictures           temp
Archiconda3-0.2.3-Linux-aarch64.sh Public            Templates
cnd.txt            redhsv.py         torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl
code              rosboard          torchvision-bak
Desktop           Rosmaster-App     trackObj.py
Documents        Rosmaster-App-bak Version.txt
Downloads        run_docker_autodrive.sh Videos
findObj.py       run_docker.sh     vision-release-0.16
maps             samples-guide.pdf yolov5-7.0
Music            software           Yolov5PtFile
jetson@jetson-desktop: $ is
4ca
jetson@jetson-desktop: $ id
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
-----
root@jetson-desktop: /#

```

iCar 镜像 docker 容器中，定义了如下快捷指令：

快捷指令	完整指令	指令用途
n1	ros2 run icar_bringup Mcnamu_driver_X3	启动小车底盘驱动 ROS 应用节点
n2	ros2 launch sllidar_ros2 sllidar_launch.py	启动小车雷达 ROS 应用节点
n3	ros2 run icar_laser laser_Avoidance_a1_X3	启动小车雷达避障 ROS 应用节点
n4	ros2 run icar_laser laser_Tracker_a1_X3	启动雷达跟随 ROS 应用节点
n5	ros2 run icar_laser laser_Warning_a1_X3	启动雷达警卫 ROS 应用节点
m1	ros2 launch astra_camera astra.launch.xml	启动深度相机 ROS 应用节点
m2	ros2 run icar_astra colorHSV	启动 HSV 颜色分析 ROS 应用节点
m3	ros2 run icar_astra colorTracker	启动颜色追踪 ROS 应用节点

8. 在 iCar 镜像的 docker 容器中执行快捷指令 n1

```

root@jetson-desktop: /
[INFO] [sllidar_node-8]: process has finished cleanly [pid 615]
root@jetson-desktop: /# exit
exit
jetson@jetson-desktop: $ t
7768
jetson@jetson-desktop: $ ls
alias.txt          Pictures           temp
Archiconda3-0.2.3-Linux-aarch64.sh Public            Templates
cnd.txt            redhsv.py         torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl
code              rosboard          torchvision-bak
Desktop           Rosmaster-App     trackObj.py
Documents        Rosmaster-App-bak Version.txt
Downloads        run_docker_autodrive.sh Videos
findObj.py       run_docker.sh     vision-release-0.16
maps             samples-guide.pdf yolov5-7.0
Music            software           Yolov5PtFile
jetson@jetson-desktop: $ is
4ca
jetson@jetson-desktop: $ id
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
-----
root@jetson-desktop: /# n1
Rosmaster Serial Opened! Baudrate=115200
X3
ttnu_link
1.0
1.0
5.0
-----create receive threading-----

```

可以看到小车底盘驱动 ROS 应用节点被启动。

9. 再启动一个 Terminal 窗口，运行指令 `id` 进入 iCa 镜像容器，并执行快捷指令 `n2`:

```

root@jetson-desktop: /
root@jetson-desktop: /
root@jetson-desktop: /
[rviz2-1] [INFO] [1747288418.558871650] [rviz2]: Message Filter dropping message: frame 'laser' at time 1747288408.535 for reason 'Unknown'
[rviz2-1] [INFO] [1747288418.859013025] [rviz2]: Message Filter dropping message: frame 'laser' at time 1747288408.793 for reason 'Unknown'
[rviz2-1] [INFO] [1747288423.373768500] [rviz2]: Setting estimate pose: Frame:map, Position(0.309484, 0.521255, 0), Orientation(0, 0, -0.164473, 0.986382) = Angle: -0.330447
[rviz2-1] [INFO] [1747288438.412174043] [rviz2]: Setting estimate pose: Frame:map, Position(0.28237, 0.497573, 0), Orientation(0, 0, -0.215607, 0.97648) = Angle: -0.434026
[rviz2-1] [INFO] [1747288446.535453479] [rviz2]: Setting goal pose: Frame:map, Position(1.17434, 0.444969, 0), Orientation(0, 0, -0.193376, 0.981125) = Angle: -0.389203
[rviz2-1] [INFO] [1747288469.175053851] [rviz2]: Setting goal pose: Frame:map, Position(1.69606, 0.21083, 0), Orientation(0, 0, -0.740807, 0.671718) = Angle: -1.66854
[INFO] [rviz2-1]: process has finished cleanly [pid 720]
root@jetson-desktop: /# ^C
root@jetson-desktop: /# jetson@jetson-desktop: $ id
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
-----
root@jetson-desktop: /# n2
[INFO] [launch]: All log files can be found below /root/.ros/log/2025-05-15-05-56-02-850483-jetson-desktop-239
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [sllidar_node-1]: process started with pid [241]
[sllidar_node-1] [INFO] [1747288562.994169554] [sllidar_node]: SLLidar running on ROS2 package SLLidar.ROS2 SDK Version:1.0.1, SLLIDAR SDK Version:2.0.0
[sllidar_node-1] [INFO] [1747288563.049499487] [sllidar_node]: SLLidar S/N: E0F7ECF0C3E09ED2A0EA98F3732F4110
[sllidar_node-1] [INFO] [1747288563.049656994] [sllidar_node]: Firmware Ver: 1.29
[sllidar_node-1] [INFO] [1747288563.049677314] [sllidar_node]: Hardware Rev: 7
[sllidar_node-1] [INFO] [1747288563.100955109] [sllidar_node]: SLLidar health status : 0
[sllidar_node-1] [INFO] [1747288563.101143305] [sllidar_node]: SLLidar health status : OK.
[sllidar_node-1] [INFO] [1747288563.332698494] [sllidar_node]: current scan mode: Sensitivity, sample rate: 8 KHz, max_distance: 12.0 m, scan frequency:10.0 Hz,

```

可以看到雷达驱动节点被启动。

10. 在 Terminal 窗口中运行指令 `n5` 启动雷达自动跟随指令:

```

10, no obstacles, go forward
10, no obstacles, go forward
9, there is an obstacle on the right, turn left
9, there is an obstacle on the right, turn left
9, there is an obstacle on the right, turn left
9, there is an obstacle on the right, turn left
9, there is an obstacle on the right, turn left
9, there is an obstacle on the right, turn left
10, no obstacles, go forward
8, there is an obstacle on the left, turn right
8, there is an obstacle on the left, turn right
4, there is an obstacle in the middle left, turn right
4, there is an obstacle in the middle left, turn right
8, there is an obstacle on the left, turn right
8, there is an obstacle on the left, turn right
9, there is an obstacle on the left, turn right
10, no obstacles, go forward
10, no obstacles, go forward
10, no obstacles, go forward
9, there is an obstacle on the right, turn left
^Croot@jetson-desktop: /# n4
inprot done
init_pid: 0.1 0.0 0.1
init_pid: 2.0 0.0 2.0
init_pid: 3.0 0.0 5.0
start it
^Croot@jetson-desktop: /# n5
inprot done
init_pid: 0.1 0.0 0.1
init_pid: 3.0 0.0 5.0
start it

```

小车启动了雷达警卫 ROS 应用节点。

11. 人在小车前面走动，小车会盯着人的方向



如果人太过靠近小车，小车会发出蜂鸣器告警声。